Computer Workshop 2: Data visualisation

Roos & Pinard

Last updated: 2024-09-17

Table of Contents

Workshop Structure

1. Formulate a research question.
2. Perform exploratory data analysis [Focus of today].
3. Identify any hidden assumptions [Focus of today].
4. Fit an appropriate model.
5. Diagnose the model and check assumptions.
6. Summarise the results.
7. Interpret and provide inferences.

We covered steps 2 and 3 in the previous workshop, but we'll expand your toolkit for doing so today using data visualisation.

Learning Objectives

1. Learn how to use the `R` package `ggplot2` to do data visualisation.
2. Consider what constitutes an effective visualisation versus an ineffective one.
3. Learn some techniques to make your figures more visually appealing to an audience.

These skills will be used throughout the course.

Introduction to `ggplot2`

`ggplot2` is an implementation of the "Grammar of Graphics," a framework for data visualisation. It allows you to build plots layer by layer, starting with a base plot and then adding additional elements such as scales, themes, and geometries. Using this framework, you can create pretty sophisticated figures with surprisingly little effort. It's such a popular tool that familiarity with `ggplot2` is sometimes asked for by employers; we've even seen job adverts from companies such as Spotify listing this an an essential criterion.

It is important to note, however, that `ggplot2` is not the only way to create visualisations. It's just an increasingly preferred one. Many of the figures you will make on this course can be near-perfectly recreated using the visualisation framework offered by `base R`. We simply prefer `ggplot2` as we feel it is a more intuitive way to create figures; indeed, nearly every single figure you have and will see on this course were produced in `ggplot2`.

Load the data

For the first part of this workshop, we'll be working with data on the Amazon River dolphin, or the boto (*Inia geoffrensis* - I wonder if the person who named the species had a friend called Geoff...). For this first interaction with this data, we'll broadly say that we're interested in seeing if there is sexual dimorphism in asymptotic length (i.e. once an animal has stopped growing) between females and males, as well as how the species respond to droughts. As always, before we do any analysis, we need to ensure there is nothing *weird* in the data. In workshop 1 you did this by using simple summary statistics (e.g. `mean()`). While doing so is useful, complimenting this with data visualisations is really powerful.

Start by downloading the data from today's practical folder, called `boto.txt`.

Go through the same steps as yesterday to load in your data; create a new script in RStudio, set your working (e.g. `setwd("H:/BI3010/workshop2")`), then load the data:

```
setwd("H:/BI3010/workshop2")
boto <- read.table("boto.txt", header = TRUE)
```

Given how easy it is to do, let's do a quick `summary()` and `str()` to see if anything stands out:

```
summary(boto)

str(boto)
```

If nothing immediately seems off, we can continue.

A simple plot

We'll build this figure up slowly, one layer at a time, to make the fundamental philosophy of `ggplot2` clear. But before we can do this, we first need to install the package (if you haven't done so already) and then load it ready to use:

```
install.packages("ggplot2") # Install the software
library(ggplot2) # Load the software
```

Once we have `ggplot2` installed and loaded, we can make a simple "ggplot":

```
ggplot()
```

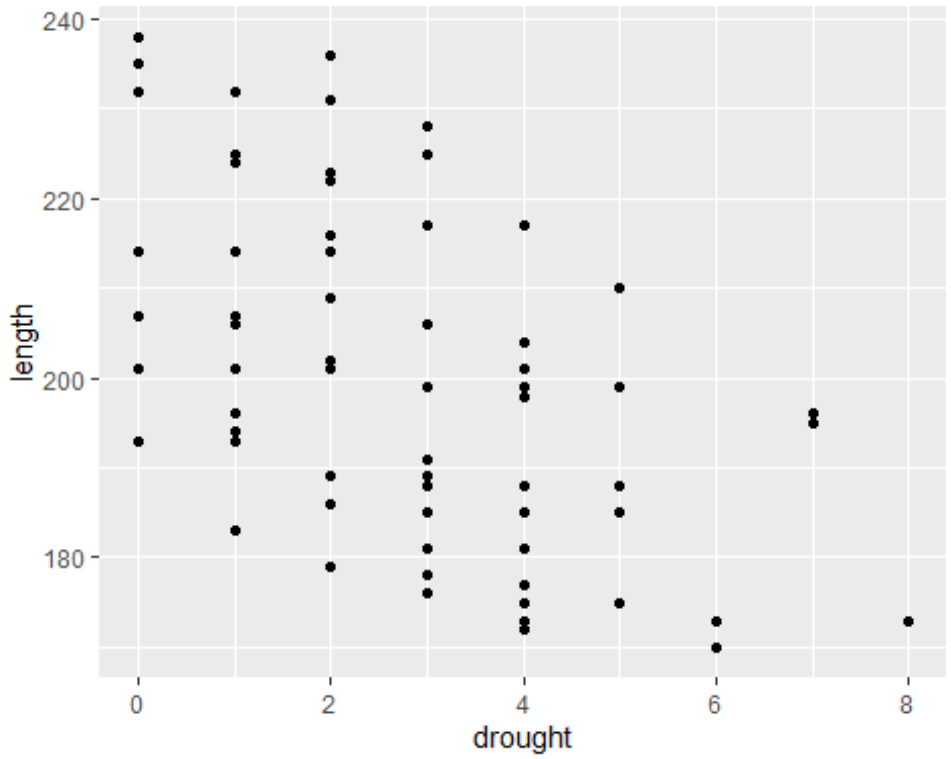

Cool… A completely empty canvas. Well, it shouldn't really come as a surprise that there's nothing in this figure. For starters, we haven't even told it which dataset to use. Let's correct that now:

```
ggplot(boto)
```

Still nothing. What we're missing here is where we tell `ggplot2` a couple of things. First, we need to say in what form we want the data to be shown. Do we want points? Or lines? Or errorbars? Secondly, we need to say which specific data, or "aesthetics" (shortened to `aes()` in `ggplot2`), we want shown and where. Do we want `length` included? On the x or y axis?

Let's have `length` be on the y-axis and `drought` be on the x-axis. And the form (or geometry) that we want this data to be shown will be as points:

```
ggplot(boto) +
  geom_point(aes(y = length, x = drought))
```



Now we're getting somewhere. From an early data analysis perspective, what have we already learnt just from the quick summary stats and from this figure?

Question Set One

1. What variables do we have available to us?

```
# 3, sex, drought and length
```

2. How many observations/samples do we have?

```
# we have data on 71 boto dolphins
```

3. Is the relationship between boto length and number of drought years likely to be positive, neutral, or negative?

```
# Very likely negative based on the figure made with:
# ggplot(boto) +
#   geom_point(aes(y = length, x = drought))
```

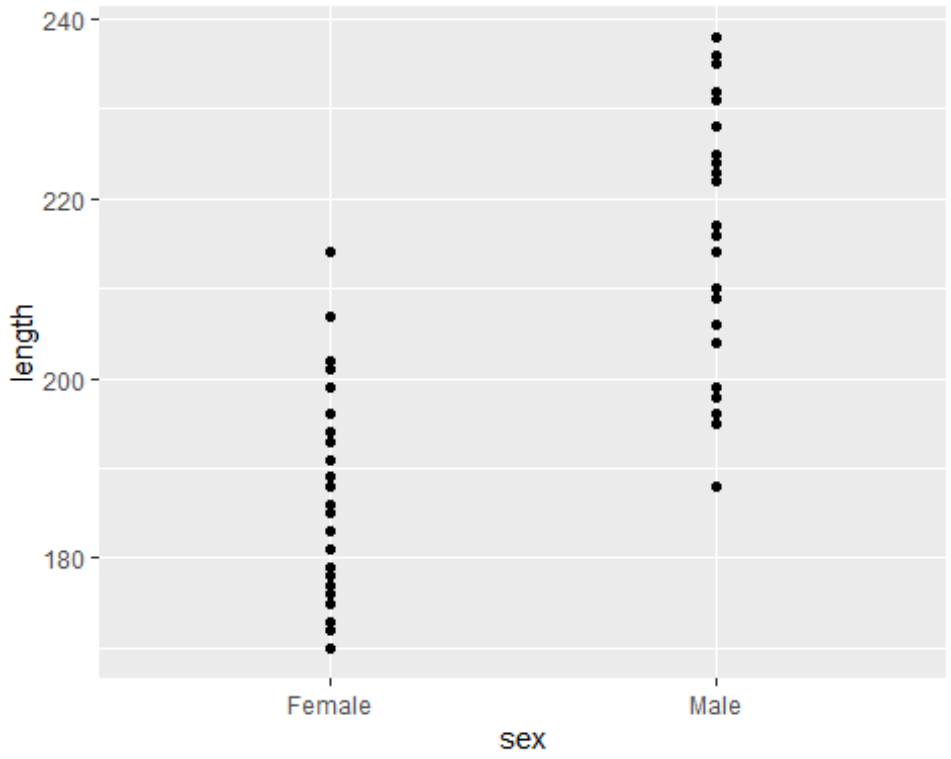
4. What range of lengths do boto dolphins reach at maturity?

```
# Using summary(), between 170.1 and 238.4
# Using the figure, roughly 170 to 240
```

Additional `geom_` layers

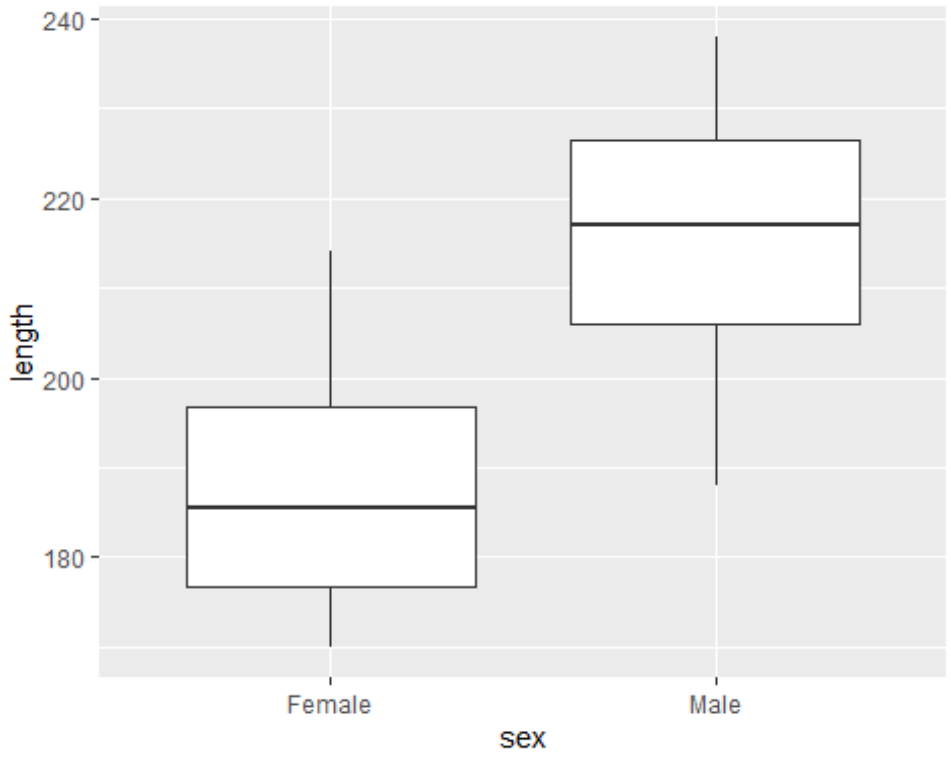
The above figure is great. It's simple and clear (though a bit ugly - we'll come back to that later), but how would we include a categorical variable like `sex`? Can we just copy our code from above and replace `drought` with `sex`? Let's see:

```
ggplot(boto) +
  geom_point(aes(y = length, x = sex))
```



Kind of? It's a bit hard to see the spread of data for each sex. A better way to show this is to use some sort of tool which will help the reader understand the range of `length` for each `sex` but also how the data is spread. There are two common options that people use for this: boxplots and violin plots. It's trivially easy to swap between these, because all we need to do is change the geometry function from `geom_point()` to either `geom_boxplot()` or `geom_violin()`:

```
# Boxplot
ggplot(boto) +
  geom_boxplot(aes(y = length, x = sex))
```




```r
# Violin plot
ggplot(boto) +
  geom_violin(aes(y = length, x = sex))
```

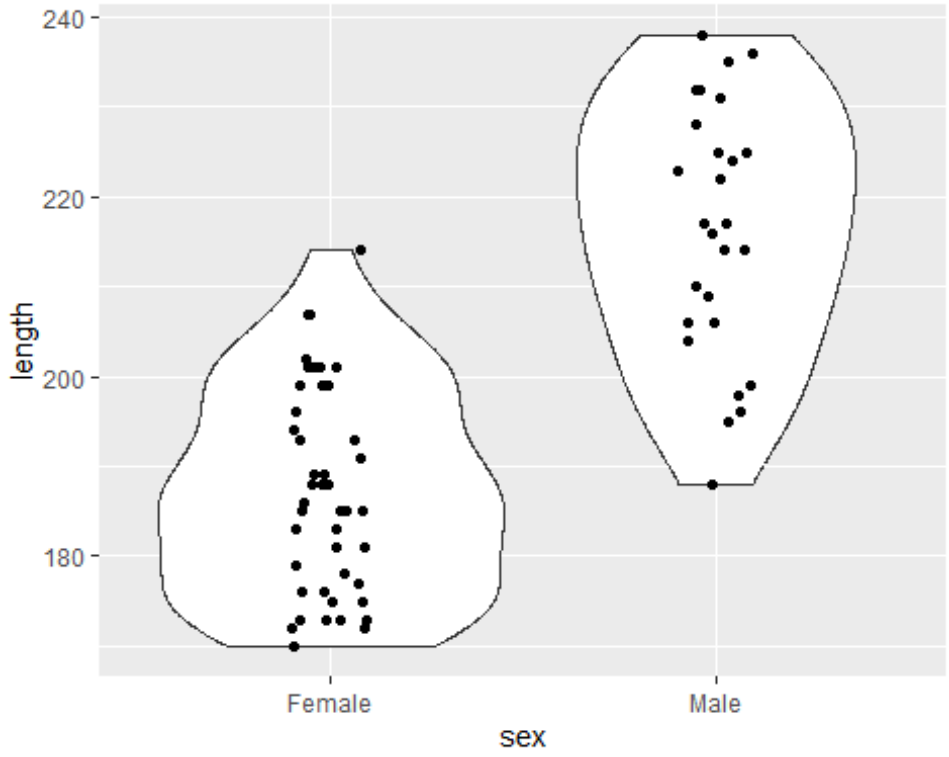
Both of these present the same information in different ways. The boxplot uses boxes and lines to indicate the range (the first and third quantiles) and the median. Conversely, the violin plot also shows the spread of data but by creating density plots which are mirrored. Boxplots have a long history and use in science, but people are increasingly using violin plots, simply because they're more transparent and clearer than boxplots.

With either boxplots or violin plots, it's fairly common that people will add data points to the figure as well - to really hammer home the spread of data. But we've seen before that using `geom_point()` basically gives us a line of points that doesn't help us understand the spread at all.

An alternative is to use something called "jiterring". When we jitter points, we add a small random value to either or both the x and y position to spread the data out a little bit. With our boto data above, we wouldn't want to add anything to y (as we're interested in the length of the dolphins, so we don't want to mess that up) but we can happily add something to x (or `sex`).

To do so, we can use `geom_jitter()` but we want to specify that "noise" should only be added to x and not y - which can can specify by saying `height = 0` (i.e. zero noise added to "height", or the y-axis), and `width = 0.1` (i.e. points can move 0.1 units to the left and right on the x-axis). Here's how we do that:

```
ggplot(boto) +
  geom_violin(aes(y = length, x = sex)) +
  geom_jitter(aes(y = length, x = sex), height = 0, width = 0.1)
```



There are two things to note with the above code. First, `geom_jitter()` comes after `geom_violin()`. If we put the violin after the jitter, then we wouldn't see any of the jittered

points; They'd be *under* the `geom_violin()` layer. Secondly, in `geom_jitter()`, we don't add the `height =` and `width =` arguments to `aes()`, as `aes()` is used to reference information within the `boto` dataset. Whenever we want to make changes to a `geom_` that are not related to the data, we do so outside of `aes()`.

Question Set Two

The questions below have issues in the code which will either cause errors or will not do what ever was intended. To help with this, try to spot the error on your own first. If you're really struggling, try running the code to see what happens.

1. What's wrong with the following code?

```
ggplot(boto) +
  geom_jitter(aes(y = length, x = sex, height = 0, width = 0.1))

# The code still runs, but with a warning saying:
# "Ignoring unknown aesthetics: height and width"
# This means height and width are specified inside the aesthetics, but they s
hould be outside
```

2. Find the error in this code:

```
ggplot(boto) +
  geom_point(aes(x = length, x = drought))

# Gives an error saying:
# "formal argument "x" matched by multiple actual arguments"
# I.e. We've specified two x-axis variables (we're missing y = )
```

3. Find the error in this code:

```
ggplot(boto) +
  geom_point(aes(x = length y = drought))

# Gives an error saying:
# "unexpected symbol in: ... geom_point(aes(x = length y"
# We're missing a comma between the x and y arguments
# Note that we're free to put x = length before y = drought
# (Technically, we're also putting our response variable on the x axis)
```

4. If I wanted to produce a histogram of boto lengths, what should my code look like? (If you're stuck, try typing `??geom_` into the console and scroll through the Help page, or type `geom_` into your script and wait for RStudio to give you a pop-up autocomplete prompt, or just do a quick google).

```
ggplot(boto) +
  geom_histogram(aes(x = length))
```

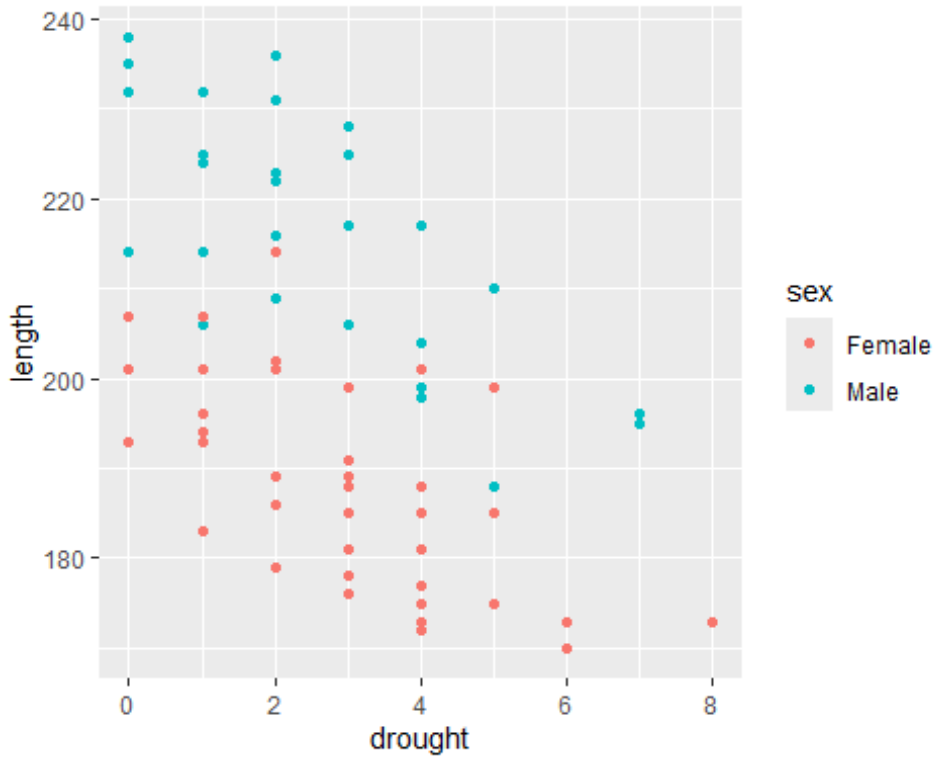
Adding more information

With these basic figures in place, we can begin expanding on them. There's no shortage available to us. We can change the colours of most `geoms_`, we can have the colour of a `geom_`

change based on another variable, the shape, the size, the transparency, and so on. More than we have the time to cover in this workshop.

With that in mind, let's use a few of these options, starting with colour. We'll keep `x = drought` and `y = length`, and colour the points according to `sex`. Because `sex` is a variable within our `boto` dataset, we'll include this within `aes()`. Remember, anything that we want to change in the figure based on a variable in our dataset, is included via `aes()`.

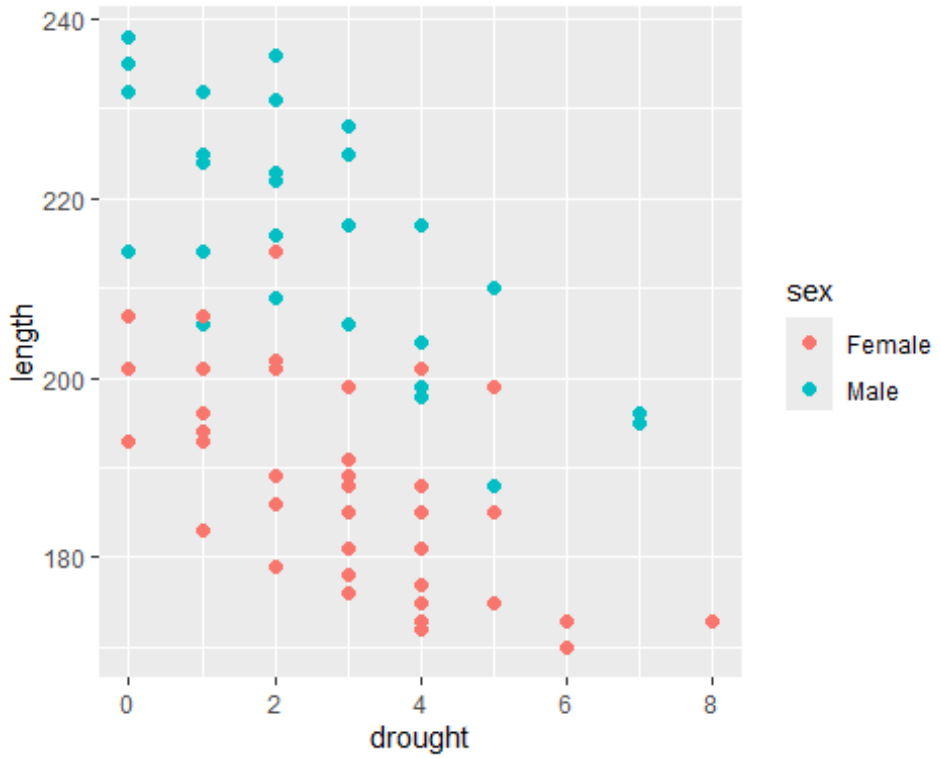
```
ggplot(boto) +
  geom_point(aes(y = length, x = drought, colour = sex))
```



With a really minor additional to the code, we now have points arbitrarily coloured red and blue for female and male. In doing so, we can see some evidence for sexual dimorphism but also that drought has a negative relationship for both sexes.

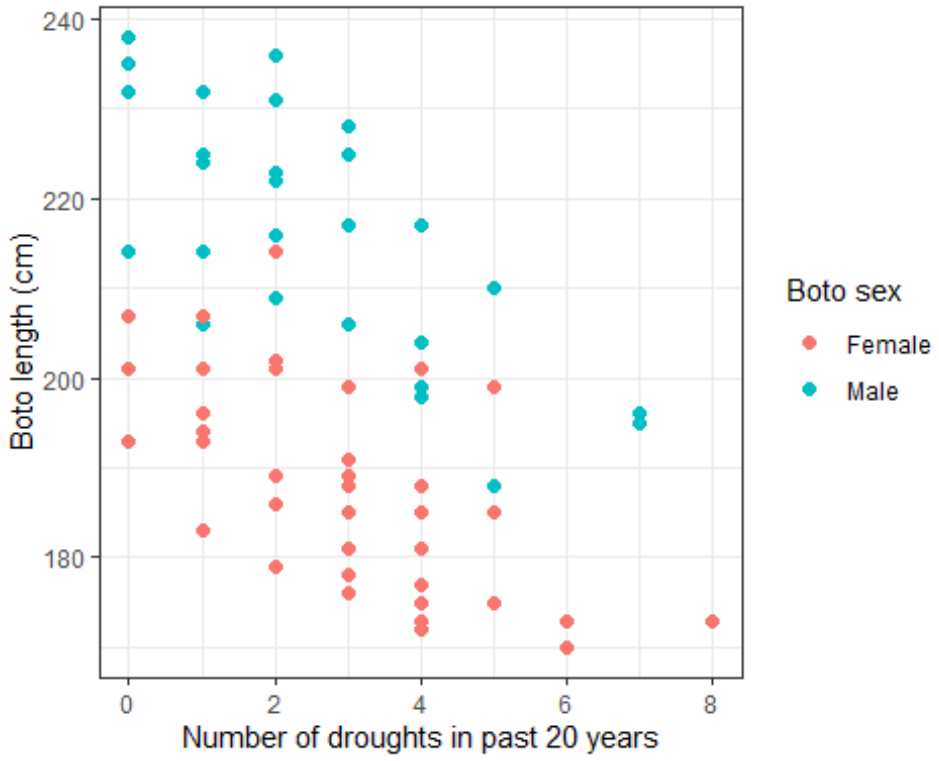
Let's assume we think the points are a bit too small for our audience to read. Well in that case, we can specify a value for `size =`, where `1.5` is the default. Should `size =` be specified within `aes()` or outside? Well, we're not using a variable in our dataset to determine the size of the point, so we include `size =` outside of `aes()`. Giving us:

```
ggplot(boto) +
  geom_point(aes(y = length, x = drought, colour = sex), size = 2)
```



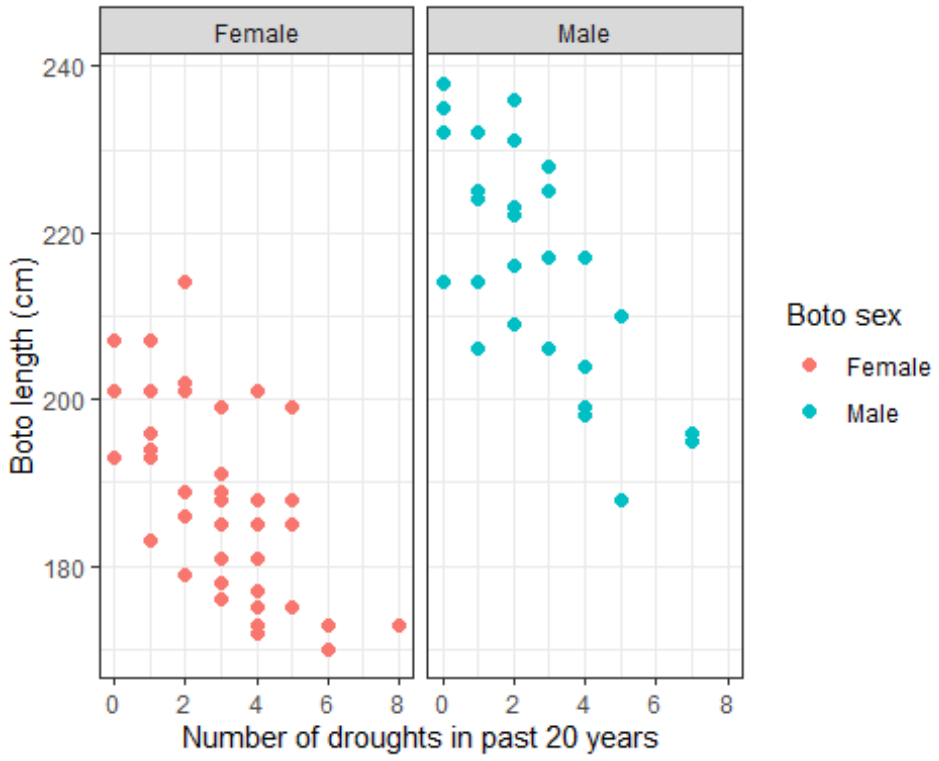
Easy enough but the figure still looks kinda ugly. Let's go about making it more visually appealing. A really easy way to do this is to use select one of many themes built into `ggplot2`. Personally, I tend to use `theme_bw()` or `theme_minimal()` but have a look online for other options if you don't like these. The next question, though, is how do we include these themes? As an additional layer. And because themes change the overall look of a figure, we can place it anywhere in our layers (though traditionally they're placed at the end). At the same time, let's change the labels to make it look less like "code" and more polished. To do so, we can create another layer using the `labs()` (short for labels) function:

```
ggplot(boto) +
  geom_point(aes(y = length, x = drought, colour = sex), size = 2) +
  labs(y = "Boto length (cm)",
       x = "Number of droughts in past 20 years",
       colour = "Boto sex") +
  theme_bw()
```



The last graphing option we'll cover in this workshop is facetting. Facetting allows us to split a figure into multiple smaller figures (technically called "multiple") based on a variable in our dataset. There are two options for facetting; `facet_wrap()` and `facet_grid()`. The difference between the two is that `facet_grid()` gives you fairly precise control, especially when you want to facet based on two variables, `facet_wrap()` will automate some of the decisions for you. Generally, `facet_wrap()` works well and is a good option to start with. In either, we need to specify how we want to split our figure into multiples, and to do so we use ~ (called "tilde"). For instance, if we use the code `facet_wrap(~sex)` then it's like we're saying, "split the figure into multiple facets based on sex". If we do that we get:

```
ggplot(boto) +
  geom_point(aes(y = length, x = drought, colour = sex), size = 2) +
  labs(y = "Boto length (cm)",
       x = "Number of droughts in past 20 years",
       colour = "Boto sex") +
  facet_wrap(~sex) +
  theme_bw()
```



Question Set Three

1. Imagine a line drawn through females and males in the faceted figure. Do you think they would have the same slope?

```
# Unlikely, male boto length looks like it has a stronger negative relationsh
ip with drought than females.
```

2. Depending on your answer to 1., do you think this might violate any of our assumptions?

```
# Yes - we're likely to break the assumption of addidivity - I.e. the length
of a boto is more than just the additive effect of sex and drought independen
tly. It looks like drought impacts males more than female. The contrast would
be that if imagined slopes were identical for both females and males.
```

3. Remember that data visualisation can be used for Exploratory Data Analysis to identify if there is anything weird in the data. Having now produced a set of figures exploring the data, do you see anything *weird*?

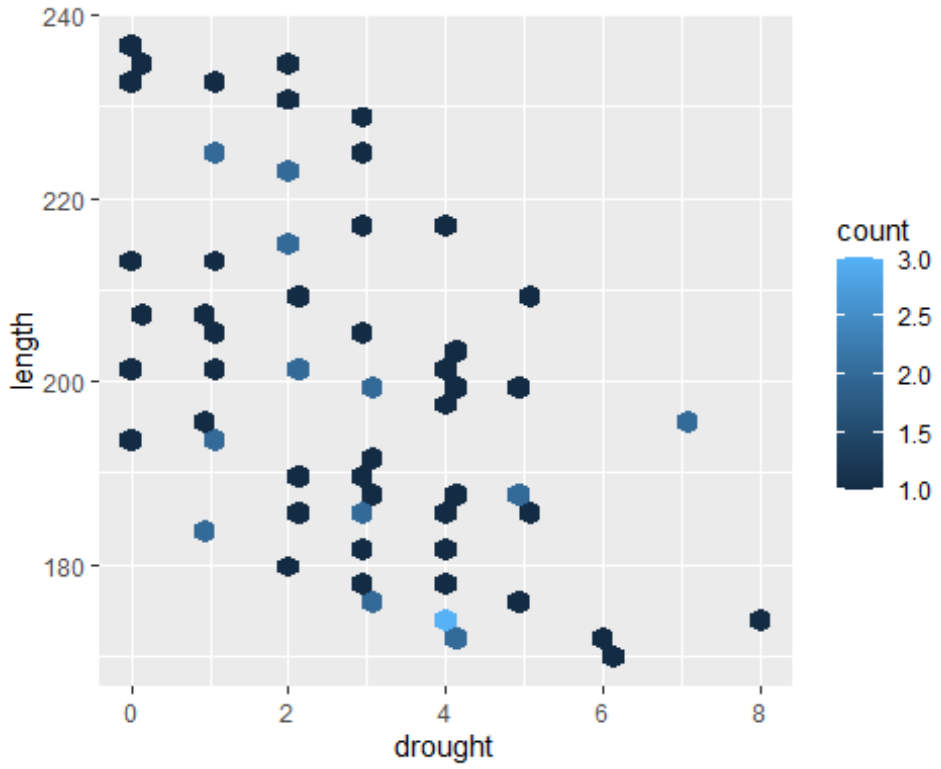
```
# No - there's nothing that jumps out as being "wrong", but it's a bit weird
how accurate length is - how do you even measure a dolphin in the Amazon? How
do you measure it to 1 cm? Something we'd want to check with our colleagues.
```

`geom_`s

Have a look here, specifically at the layers section. Using the information in the link, see if you can figure out which `geom_` to use such that you recreate the following figure. There are two things to note here. First this figure is created with a single `geom_` with only `x =` and `y =` specified, and no other options. Secondly, if you figure out which `geom_` to use, you will be

prompted to install a new package when you run the code (it's fine to install this new package - simply type 1 and enter in the console when prompted) and you may be asked "Do you want to install from sources the package which needs compilation?", if so just say no.

```
ggplot(boto) +
  geom_hex(aes(x = drought, y = length))
```



The good and the bad

We're going to do something that's, hopefully, fun but also instructive. On MyAberdeen, you find a dataset called `abdn_rent.txt`. This dataset contains > 100 records (i.e. observations or samples) of rent prices for properties listed in Aberdeenshire on the ASPC website. In addition to the rent price, 11 additional variables were recorded as well. The variables are:

1. `price` - the rent of the listed property
2. `beds` - the number of bedrooms in the property
3. `living` - the number of living rooms in the property
4. `baths` - the number of bathrooms
5. `sqmt` - the total floor area of the property (measured in m^2)
6. `epc` - th Energy Performance Certificate (A is highly energy efficient, G is terrible)

7. `tax` - the council tax band (which is based on the value of the property on 1/4/1991, A is the lowest council tax, H is the highest possible)
8. `date` - the date when the record was *added to the dataset* (note this is not when the property was listed on the site)
9. `link` - web link for the property listing
10. `house` - the property name or address
11. `lat` - the latitude coordinates of the property (note latitude is North to South - think of a "ladder", sounds kinda like "latter" going up)
12. `lon` - the longitude coordinates of the property (note longitude is East to West - think of "the *long* way around")

Spend a bit of time familiarising yourself with the data, i.e. do a quick EDA to make sure there's nothing weird. If you find what you think is a mistake, well in this case, you can check and correct yourself, as you have the *original* data because of the web link! We'll come back to this dataset later in the course where you'll be using it to predict how much rent a landlord *should* be asking for rent, so it's not wasted time getting to know the data.

Once you're happy you know the data relatively well, let's get to the tasks.

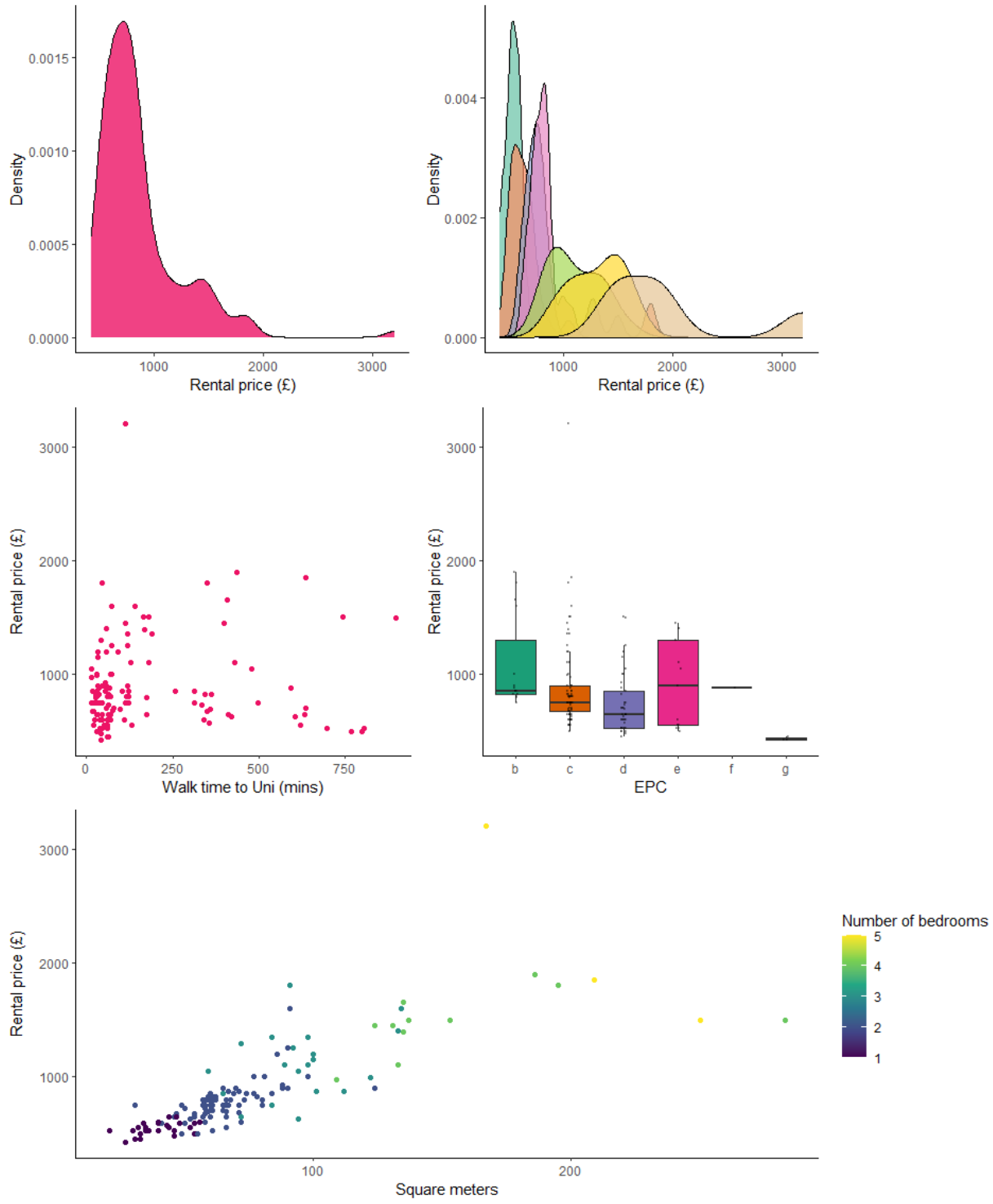
The good

Using whichever variables you like, make the most informative and visually appealing figure you can. Imagine you have been hired by a real estate company to create a figure that helps potential renters understand what may be determining rent prices, using `R`, `ggplot2` and the `abdn_rent.csv` dataset.

Create your most informative and visually appealing figure now - note that your "audience" here are not scientists, they're members of the public, so we want something that effectively communicates the information but is also eye catching. Keep the lecture on effective communication in your mind and avoid some of the pitfalls we discussed. If you need help figuring out how to implement one of your ideas, see Intro2R chapter for a slightly more detailed walk through on `ggplot2`, or else use your LLM of choice to help. If using an LLM, just remember to mention that you are using `R`, `ggplot2` and do not want to use any additional packages. There is nothing wrong with using additional packages, but they can just add lots of extra moving parts to the mix (and cause conflict issues). We'd suggest you try and limit the number of additional packages you have loaded while you're still getting the hang of things, but if you feel comfortable using `R`, then go ahead by all means.

```
abdn <- read.table("abdn_flats.txt", header = TRUE)

# Here's my version of the best looking and most informative figure using the
abdn flat data
p1 <- ggplot(abdn) +
  geom_density(aes(x = price), fill = "#EC1365", alpha = 0.8) +
  scale_fill_brewer(palette = "Dark2") +
  theme_classic() +
```

```
  labs(x = "Rental price (£)",
       y = "Density")

p2 <- ggplot(abdn) +
  geom_density(aes(x = price, fill = tax), alpha = 0.7, show.legend = FALSE)
+
  scale_fill_brewer(palette = "Set2") +
  theme_classic() +
  labs(x = "Rental price (£)",
       fill = "Tax\nband",
       y = "Density")

p3 <- ggplot(abdn) +
  geom_point(aes(x = commute_time_minutes, y = price), colour = "#EC1365") +
  theme_classic() +
  labs(x = "Walk time to Uni (mins)",
       y = "Rental price (£)")

p4 <- ggplot(abdn) +
  geom_boxplot(aes(x = epc, y = price, fill = epc), outliers = FALSE,
               show.legend = FALSE) +
  geom_jitter(aes(x = epc, y = price),
              alpha = 0.3, size = 0.5, height = 0, width = 0.05) +
  scale_fill_brewer(palette = "Dark2") +
  theme_classic() +
  labs(x = "EPC",
       y = "Rental price (£)")

p5 <- ggplot(abdn) +
  geom_point(aes(x = sqmt, y = price, colour = beds)) +
  theme_classic() +
  scale_colour_viridis_c() +
  labs(x = "Square meters",
       colour = "Number of bedrooms",
       y = "Rental price (£)")

library(patchwork) # Patchwork allows you to "stitch" ggplots together

## Warning: package 'patchwork' was built under R version 4.3.3

(p1 + p2) / (p3 + p4) / p5
```



The bad

Now for the part I'm excited about. Do the same as above, but this time, make it a miserably ugly figure. It still needs to *contain* the information (i.e. no blank figures) and be *somewhat*

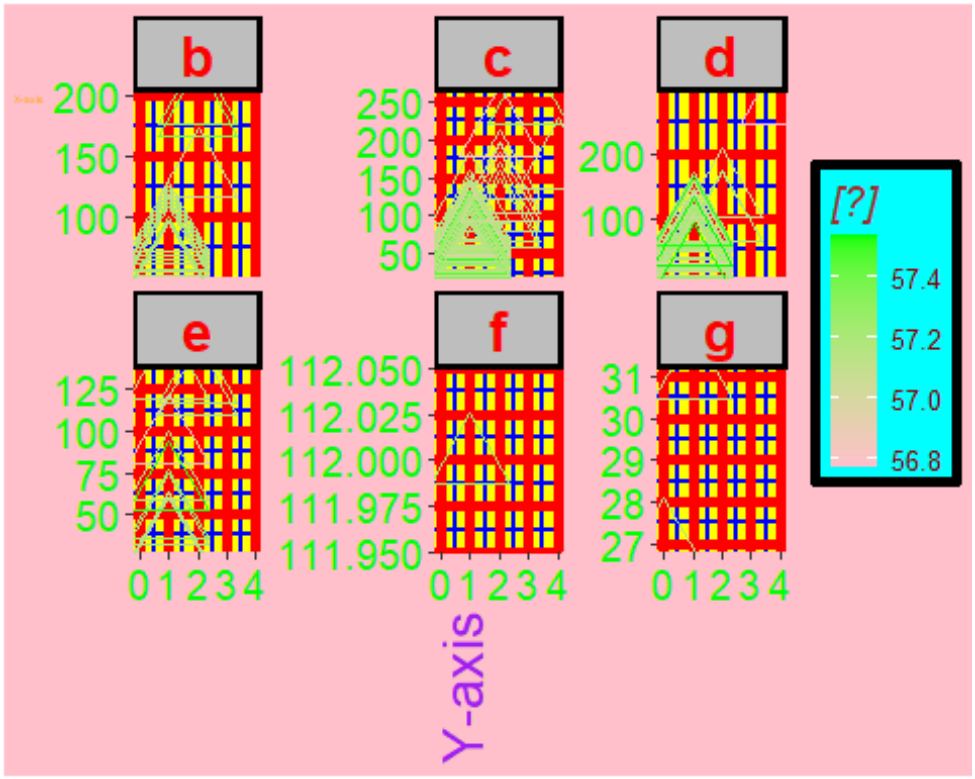
readable. If you need inspiration, search for “graph crimes”, or “chart crimes” in your search engine - there are *lots* of examples you can use for “inspiration”.

Once you’re happy with your masterpieces, add your figures onto the MyAberdeen discussion board, and we’ll have an “Art Exhibit” to finish the workshop :D.

```
# Here's my "best" attempt at making something disgusting
# Most of this is done by messing around with theme()
ggplot(abdn) +
  geom_point(aes(x = living, y = sqmt, colour = lat), shape = 24, size = 10,
alpha = 1) +
  theme(
    panel.background = element_rect(fill = "yellow"),
    plot.background = element_rect(fill = "pink"),
    panel.grid.major = element_line(colour = "red", size = 2),
    panel.grid.minor = element_line(colour = "blue", size = 1),
    axis.title.x = element_text(size = 20, angle = 90, vjust = 0.5, colour =
"purple"),
    axis.title.y = element_text(size = 4, angle = 0, hjust = 0.5, colour = "o
range"),
    axis.text = element_text(size = 15, colour = "green"),
    legend.background = element_rect(fill = "cyan", colour = "black", size =
2),
    legend.title = element_text(size = 15, colour = "brown", face = "italic")
,
    legend.text = element_text(size = 10, colour = "darkred"),
    strip.background = element_rect(fill = "grey", colour = "black", size = 1
.5),
    strip.text = element_text(size = 20, colour = "red", face = "bold")
  ) +
  scale_colour_gradient(low = "pink", high = "green") +
  labs(x = "Y-axis",
       y = "X-axis",
       colour = "[?]") +
  facet_wrap(~epc, scales = "free_y")
```

```
## Warning: The `size` argument of `element_rect()` is deprecated as of ggplo
t2 3.4.0.
## ℹ Please use the `linewidth` argument instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Warning: The `size` argument of `element_line()` is deprecated as of ggplo
t2 3.4.0.
## ℹ Please use the `linewidth` argument instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```


b
c
d
e
f
g
200
150
100
250
200
150
100
50
200
100
125
100
75
50
112.050
112.025
112.000
111.975
111.950
31
30
29
28
27
0 1 2 3 4
[?]
57.4
57.2
57.0
56.8
Y-axis